Predicting Human-Perceived Coding Difficulty from Source-Code Features Using Regression: A Student-Rating Study

MUHAMMAD REHAN, ISSA THERASE DELSON, and KHADAR BABU SK

School of Advanced Sciences, Vellore Institute of Technology, Vellore, Tamil Nadu, India

Correspondence: Dr. Khadar Babu SK

khadarbabu.sk@vit.ac.in
rehanpeetikathazha@gamil.com
issatherase@gmail.com

ABSTRACT

ntroduction: While classical metrics like McCabe's cyclomatic complexity have guided software engineering for decades, they were designed for production codebases rather than learners. Whether these formulas reflect what a novice programmer actually struggles with remains debatable. In this study, we tested whether students' subjective difficulty can be predicted from basic, syntactic code counts, which structural features carry real weight, and whether cyclomatic complexity provides any useful signal once simple line and branch counts are known**.**

Methods: We gathered 100 self-contained Python snippets ranging from simple math scripts to recursive algorithms. For each, we extracted seven surface features: lines of code, nesting depth, loop counts, conditionals, functions, variable names, and cyclomatic complexity. Thirty-two computing students rated each snippet's difficulty (1 to 10) and their own confidence (1 to 5) across four balanced forms, giving 832 total ratings (8 ratings per snippet). After running test-retest checks and intraclass correlations, we trained OLS linear regression, ridge, lasso, elastic net, and random forest models using repeated 5-fold cross-validation against baseline predictors.

Results: Intra-rater test-retest difference averaged just 0.94 points, with no student showing erratic swings over 3 points. Aggregated rating reliability reached ICC$(1, k) = 0.94$. Linear and regularised models accurately tracked student ratings (Linear: RMSE $= 0.685 \pm 0.129$, $R^2 = 0.877 \pm 0.061$; Elastic Net: RMSE $= 0.689 \pm 0.120$; Ridge: RMSE $= 0.690 \pm 0.123$; Lasso: RMSE $= 0.690 \pm 0.122$), cutting baseline error by 67% and outperforming Random Forest (RMSE $= 0.837$). Nesting depth was the strongest predictor by a wide margin (ridge $\beta = 0.89$), followed by lines of code (0.47), loops (0.42), and variables (0.38). Cyclomatic complexity yielded zero independent predictive gain because it is mathematically collinear with loop and branch counts (VIF $> 10^{14}$).

Discussion: Control-flow nesting and snippet length govern how difficult code feels to learners. Standalone counts of loops and conditions explain student perceptions completely, rendering McCabe's composite metric redundant in this setting. Transparent, regularised linear models predict difficulty reliably, offering an easy way for instructors and auto-graders to sequence problems without computing complex software indices**.**

Keywords: code difficulty; perceived complexity; cyclomatic complexity; regression analysis; regularised regression; programming education; novice programmers

1 INTRODUCTION

Computer science instructors often struggle to gauge how hard a coding task will feel to beginners before assigning it. When problems are sequenced poorly, novices get stuck, lose confidence, and disengage. Software engineering has tried for decades to quantify program difficulty through structural code metrics. McCabe's cyclomatic complexity counts independent control paths [1], Halstead focused on token counts and operator frequency [2], and Campbell designed Cognitive Complexity specifically to penalise broken reading flows [3].

Other researchers took an empirical route by collecting subjective readability scores directly from programmers. Buse and Weimer trained machine learning models on snippet ratings from developers [4], an approach later simplified with leaner feature sets [5] and lexical cues [6]. Even so, later studies observed that established metrics frequently fail to predict whether a programmer actually understands what a function does [7]. A recurring complication is code length: because most complexity metrics correlate strongly with raw lines of code, it is difficult to know if a metric captures real cognitive difficulty or simply program size [8].

Three questions left open in the literature prompted our work. First, most published rating sets recruit experienced developers and focus on broad readability rather than novice-perceived difficulty, leaving computer science education without clear empirical benchmarks. Second, earlier studies often collapsed human ratings into coarse binary categories, discarding the fine-grained variance needed for regression models. Third, basic syntactic counts correlate heavily with one another, making standard OLS estimates fragile; whether cyclomatic complexity adds any unique predictive insight beyond raw lines, loops, and branches has rarely been evaluated under regularised conditions.

Our contribution is purely empirical. We are not introducing a new software metric or inventing a novel algorithm. Instead, we collected a controlled set of student ratings, ran thorough rater-reliability and multicollinearity diagnostics, and evaluated standard regularised models using repeated cross-validation. With 100 snippets, we prioritized simple, interpretable models with robust bootstrap uncertainty intervals. We address three specific questions:

- **RQ1:** Which source-code features dominate students' perceptions of coding difficulty?
- **RQ2:** Does cyclomatic complexity improve predictions when basic loop, branch, and line counts are already included?
- **RQ3:** Which regression family—linear, penalised, or decision trees—yields the lowest error under cross-validation?

2 METHODOLOGY

2.1 Snippet Corpus

A corpus of 100 short Python snippets (S001 to S100) was assembled from introductory programming curricula, algorithm repositories, and textbook problem sets. Snippets were designed to cover a broad spectrum of structural complexity, ranging from basic sequential input-output scripts and elementary mathematical routines to complex multi-branch control flow, deeply nested loops, recursion, and dynamic programming formulations. Snippets were self-contained, stripped of comments and docstrings, and formatted consistently according to PEP 8 standards to ensure that ratings reflected code structure rather than extraneous visual cues. Each snippet was short enough to be thoroughly inspected within approximately one to two minutes.

2.2 Feature Extraction

Seven structural code features were extracted automatically for each snippet using Python's built-in abstract syntax tree (ast) module and the radon library (Table 1). Cyclomatic complexity was determined in accordance with McCabe's definition as the number of decision points plus one (decision points including if, elif, for, while, and boolean operations). The automated feature extractor script was verified against a manual count performed on 10 randomly selected snippets, yielding 100% agreement across all metric dimensions.

2.3 Collection of Difficulty Ratings

Difficulty ratings were gathered through four standardized Google Forms questionnaires (Forms A, B, C, and D), corresponding to four balanced snippet subsets of 25 snippets each, stratified by cyclomatic complexity to guarantee equal complexity distributions. Each questionnaire contained 26 rating tasks: the 25 distinct snippets for that form, plus a repetition of the first snippet presented at the very end to evaluate intra-rater consistency. A total of 32 undergraduate and postgraduate computing students participated (8 participants per form), providing exactly 8 independent evaluations for every snippet (832 total rating submissions). For each snippet, participants rated perceived understanding difficulty on a 10-point Likert-type scale (1 = very easy to 10 = very hard) and indicated their comprehension confidence on a 5-point scale (1 = very unconfident to 5 = highly confident). Students were instructed to read the code carefully without running the interpreter or consulting external references. Participation was voluntary, uncompensated, and anonymized using randomly assigned 4-digit participant codes under institutional ethical clearance.

2.4 Data Cleaning

Ratings were converted to long format, with one row per student and snippet. Rows were removed if consent was not given, if the difficulty rating was missing or outside 1 to 10, if the confidence rating was outside 1 to 5, if the participant code was not in the specified format, if the snippet identifier did not exist in the corpus, or if the same student rated the same snippet more than once apart from the planned repeat (the first submission was retained). The number of rows removed under each rule was recorded and is reported in Table 2.

2.5 Reliability Assessment

Two reliability assessments were conducted. First, intra-rater consistency was evaluated by computing the absolute score difference between the first and repeated presentation of the verification snippet for each student; participants displaying a gap exceeding 3 points were flagged for sensitivity screening. Second, inter-rater reliability across participants was quantified using the one-way random-effects intraclass correlation coefficient, ICC(1) for individual ratings and ICC(1,k) for the average rating across k raters [9, 10], appropriate for designs where different rater subsets evaluate different items.

2.6 Construction of the Modelling Table

The target variable was the mean difficulty rating of each snippet, computed from the first (non-repeat) ratings only. The modelling table contained one row per snippet, the seven features as independent variables and the mean rating as the dependent variable. Snippet identifier, form and number of ratings were excluded from the predictors to avoid information leakage. Averaging was used because it reduces rater-specific noise and gives one observation per snippet.

2.7 Exploratory Analysis and Outlier Screening

The distribution of the target was inspected with a histogram, each feature was plotted against the target, and Pearson and Spearman correlation matrices were computed. Features and the target were screened using the interquartile range criterion, with limits at 1.5 times the interquartile range beyond the quartiles. Snippets flagged by the screening were inspected individually. Extraction errors were corrected, genuine cases were retained, and no observation was removed solely because it reduced model fit. Influential observations in an ordinary least squares fit were identified with Cook's distance [11].

2.8 Multicollinearity Diagnostics

Because cyclomatic complexity is structurally determined by decision points (loops and conditions), substantial collinearity was anticipated. Multicollinearity was evaluated using Variance Inflation Factors (VIF), where VIF values exceeding 5 to 10 signify problematic multicollinearity that degrades ordinary least squares parameter stability [12].

2.9 Regression Models

Five regression architectures were evaluated: Ordinary Least Squares (OLS) linear regression, Ridge regression (L2 penalisation), Lasso regression (L1 penalisation and feature selection), Elastic Net (combined L1/L2 penalties), and Random Forest regression (an ensemble of 300 decision trees with min_samples_leaf = 2). Hyperparameter optimisation for regularisation parameters (alpha and l1_ratio) was conducted via internal cross-validation nested strictly inside each training fold.

2.10 Baselines and Validation

Four reference baselines were incorporated: an intercept-only (dummy) baseline predicting the training mean, a single-predictor LOC model, a single-predictor cyclomatic complexity model, and a linear model containing all features except cyclomatic complexity (directly answering RQ2). To eliminate split-selection bias, all models were evaluated using repeated five-fold cross-validation (5 repeats = 25

distinct validation folds). Standardisation was fitted exclusively on training folds to prevent information leakage [17].

2.11 Performance Metrics

Predictive performance was measured with the root mean squared error (RMSE), the mean absolute error (MAE), the coefficient of determination (R-squared) and the Spearman rank correlation between observed and out-of-fold predicted ratings. Metrics are reported as the mean and standard deviation across folds. Because folds are not independent, differences between models were interpreted descriptively, and models whose results overlapped within one standard deviation were treated as comparable.

2.12 Interpretation of Feature Influence

Feature influence (RQ1) was quantified using standardised regression coefficients from Ridge and Lasso models fitted to the dataset, accompanied by 95% bootstrap confidence intervals computed over 1,000 snippet-level resamples, and permutation feature importance from the Random Forest evaluated on held-out snippet folds. Agreement across parametric and non-parametric importance rankings was evaluated.

2.13 Sensitivity Analyses

Four sensitivity analyses were conducted: repeating the main analysis after excluding students flagged by the repeat-snippet check, evaluating model sensitivity to using the median instead of the mean rating as the target, repeating the analysis after removing influential observations with large Cook's distance, and fitting a linear mixed-effects model on the rating-level data with a random intercept for student to separate rater-level clustering from snippet-level effects.

2.14 Software Environment

Statistical analyses and machine learning pipelines were implemented in Python 3.14.6 using pandas 3.0.3 and NumPy 2.5.1 for data structures, the ast module and radon for structural metric parsing, scikit-learn 1.9.0 [19] for machine learning pipelines and cross-validation, statsmodels 0.15.0 [20] for regression diagnostics and mixed-effects modelling, and Matplotlib 3.11.1 for visualization.

3 RESULTS

All reported values are derived directly from the empirical dataset and statistical analyses.

3.1 Dataset Verification and Cleaning

A total of 32 students provided informed consent and completed the questionnaire forms, generating 832 raw rating entries (800 unique snippet ratings and 32 repeat-snippet check ratings). Application of the quality verification protocol confirmed that zero entries contained missing difficulty scores, out-of-bounds values (difficulty outside 1-10 or confidence outside 1-5), invalid participant codes, unrecognized snippet IDs, or unprompted duplicate submissions. The complete cleaning log is presented in Table 2. The final modelling table comprised all 100 snippets, each evaluated by exactly 8 independent student raters (mean = 8.0, range = 8 to 8 ratings per snippet).

3.2 Reliability of Ratings

Intra-rater reliability was notably high: the mean absolute difference between first and repeated ratings across all 32 students was 0.94 points (SD = 0.91, range = 0 to 3). Exactly 13 students exhibited zero difference (gap = 0), 11 differed by 1 point, 5 differed by 2 points, and 3 differed by 3 points. Zero students (0.0%) exceeded the 3-point inconsistency threshold. Inter-rater reliability evaluated via the one-way random-effects model yielded an intraclass correlation coefficient of ICC(1) = 0.67 (95% CI [0.60, 0.74]) for single ratings, denoting moderate individual-level reliability. For the snippet-aggregated target (mean of k = 8 ratings), reliability reached ICC(1,k) = 0.94 (95% CI [0.92, 0.96]), establishing excellent measurement reliability for the regression target.

3.3 Descriptive Statistics and Correlations

Descriptive statistics for the seven structural predictors and snippet mean difficulty ratings across all 100 snippets are summarised in Table 3. Snippet mean difficulty spanned from 1.500 to 9.375 (mean = 5.258, SD = 2.068, median = 5.125, IQR = 3.125). Program size ranged from 3 to 39 lines of code (mean = 20.02, SD = 9.82), nesting depth from 0 to 4 levels (mean = 1.97, SD = 1.23), and cyclomatic complexity from 1 to 9 (mean = 4.65, SD = 2.15). All seven features demonstrated statistically significant positive correlations with mean difficulty: lines of code (Pearson $r = 0.887$, $p < 0.001$; Spearman rho = 0.890, $p < 0.001$) and nesting depth ($r = 0.886$, $p < 0.001$; rho = 0.887, $p < 0.001$) exhibited the strongest associations, followed by cyclomatic complexity ($r = 0.844$, $p < 0.001$), variable count ($r = 0.826$, $p < 0.001$), loop count ($r = 0.758$, $p < 0.001$), conditions ($r = 0.690$, $p < 0.001$), and functions ($r = 0.614$, $p < 0.001$). Outlier screening using the 1.5 × IQR fence identified zero outlying snippets across any feature or the target variable. Cook's distance diagnostics identified six observations exceeding the 4/N = 0.04 guideline (S015, S022, S032, S042, S056, S092), with S056 having the largest value (D = 0.419). Inspection confirmed that all cases represented valid code and genuine ratings; none exceeded the conventional influence cutoff of 1.0, and all were retained.

3.4 Multicollinearity

Variance inflation factors are presented in Table 4. When all seven structural predictors were entered into the regression model simultaneously, loops, conditions, and cyclomatic complexity yielded infinite VIF values ($> 10^{14}$), reflecting an exact rank deficiency in the design matrix (rank 7 of 8 including the intercept). This occurred because cyclomatic complexity is mathematically defined as loops + conditions + 1 across all snippets. When cyclomatic complexity was excluded, the VIF values for the remaining six features dropped to well within safe boundaries: LOC was 5.92, nesting depth was 3.45, variables was 3.96, conditions was 2.59, loops was 2.13, and functions was 1.92. All remaining features fell below the conservative multicollinearity threshold of 10.

3.5 Model Comparison

Cross-validated predictive performance across 25 folds (repeated 5-fold CV with 5 repeats) is detailed in Table 5. Ordinary least squares linear regression and elastic net achieved the highest overall accuracy (Linear: RMSE = 0.685 ± 0.129, MAE = 0.533 ± 0.082, $R^2 = 0.877 \pm 0.061$, Spearman rho = 0.928 ± 0.031; Elastic Net: RMSE = 0.689 ± 0.120, $R^2 = 0.877 \pm 0.057$), closely followed by lasso (RMSE = 0.690 ± 0.122, $R^2 = 0.876 \pm 0.057$) and ridge (RMSE = 0.690 ± 0.123, $R^2 = 0.875 \pm 0.061$). Compared to the intercept-only baseline (RMSE = 2.061 ± 0.186, $R^2 = -0.034 \pm 0.040$), the linear models achieved a 67% reduction in

prediction error. Performance differences between the linear and regularised variants were negligible (< 0.005 in RMSE) and well within one fold-level standard deviation. Crucially, all linear models substantially outperformed Random Forest (RMSE = 0.837 ± 0.103, MAE = 0.690 ± 0.081, R^2 = 0.822 ± 0.060), confirming that parsimonious linear models generalise better than complex tree ensembles for this sample size.

3.6 Role of Cyclomatic Complexity

In direct response to RQ2, the full linear model containing cyclomatic complexity produced predictive metrics identical to the model excluding cyclomatic complexity (both models: RMSE = 0.685 ± 0.129, MAE = 0.533 ± 0.082, R^2 = 0.877 ± 0.061, Spearman rho = 0.928 ± 0.031). Furthermore, when tested as solitary univariate predictors, lines of code (LOC only: RMSE = 0.957 ± 0.105, R^2 = 0.769 ± 0.066) substantially outperformed cyclomatic complexity (Cyclomatic only: RMSE = 1.121 ± 0.124, R^2 = 0.674 ± 0.139). These findings demonstrate that classical cyclomatic complexity provides zero marginal predictive value over simpler structural features and is a notably inferior standalone proxy compared to code length.

3.7 Feature Influence

Standardised coefficients and permutation importances are reported in Table 6. Standardised coefficients from ridge and lasso regression across 1,000 bootstrap resamples revealed that nesting depth exerted the strongest effect on perceived difficulty (Ridge: 0.89, 95% CI [0.69, 1.07]; Lasso: 0.90, 95% CI [0.67, 1.12]), indicating an approximate 0.9-point increase in difficulty per 1 SD increase in nesting depth. Lines of code (Ridge: 0.47 [0.23, 0.69]; Lasso: 0.46 [0.12, 0.69]), loop count (Ridge: 0.42 [0.23, 0.62]; Lasso: 0.39 [0.19, 0.61]), and variable count (Ridge: 0.38 [0.15, 0.61]; Lasso: 0.36 [0.11, 0.59]) also exhibited strong, strictly positive effects across 100% of bootstrap resamples. In contrast, conditions and functions yielded near-zero coefficients (Lasso retained conditions in only 44% of resamples and functions in 24%). Random forest permutation importance confirmed this hierarchy, ranking lines of code highest (mean RMSE increase = 0.777 ± 0.190), followed by student programming experience (0.239 ± 0.070) and nesting depth (0.210 ± 0.117).

3.8 Sensitivity Analyses

Sensitivity analyses confirmed the robustness of all conclusions across multiple operationalisations. First, because no raters exceeded the 3-point repeat gap threshold, overall data quality was verified; excluding the three participants with the maximum observed gap of 3 points (N = 29 raters) produced identical model performance (RMSE = 0.682 ± 0.127, R^2 = 0.878 ± 0.058). Second, utilising the median difficulty rating as the target produced nearly identical predictive fit (RMSE = 0.826 ± 0.167, R^2 = 0.840 ± 0.076, Spearman rho = 0.921 ± 0.038; Pearson r with mean target = 0.989) with identical coefficient rankings (nesting depth = 0.924, variables = 0.512, LOC = 0.467, loops = 0.451). Third, omitting influential snippet S056 (Cook's D = 0.419, N = 99 snippets) further enhanced performance (RMSE = 0.627 ± 0.086, R^2 = 0.893 ± 0.055) without altering feature rankings or signs. Removing all six snippets exceeding 4/N Cook's distance yielded RMSE = 0.565 ± 0.108 and R^2 = 0.912 ± 0.040. Finally, a linear mixed-effects model on the 800 individual ratings confirmed significant fixed effects for nesting depth (B = 0.783, $p < 0.001$), LOC (B = 0.044, $p < 0.001$), loops (B = 0.399, $p < 0.001$), variables (B = 0.161, $p < 0.001$), and student experience level (B = -0.605, $p < 0.001$), while conditions and functions remained non-significant, confirming complete stability across both aggregate and multilevel models.

4 DISCUSSION

Looking across our results, the answers to our three questions are clear. First (RQ1), nesting depth and overall line count are the main reasons code feels hard to students, with loop and variable counts playing secondary roles; individual condition statements and function counts had almost no measurable effect. Second (RQ2), cyclomatic complexity failed to provide any marginal predictive value over elementary structural counts because it is a direct linear combination of loops and branches. Third (RQ3), standard regularised linear models (OLS, Ridge, Lasso, and Elastic Net) predicted student ratings accurately ($R^2 = 0.877$,RMSE $= 0.685$), easily beating non-linear Random Forest ensembles.

These findings match what cognitive psychology tells us about working memory in programming. Deeply nested blocks force a student to keep multiple conditional contexts and variable states in mind simultaneously. Raw code length acts similarly: reading longer functions taxes attention simply because there is more program state to track.

The fact that McCabe's metric provided zero unique predictive value carries direct implications for teaching. Introductory courses and automated grading platforms frequently cite cyclomatic complexity as a universal benchmark for difficulty. In our experiments, however, it added no information beyond what was already captured by separate counts of loops and conditionals. In fact, as an isolated metric, cyclomatic complexity proved significantly worse than simple line count. This lines up with Gil and Lalouche's critique that many software metrics are largely proxies for code size [8].

Our results also suggest that complex non-linear models are unnecessary here. Random Forest overfitted the smaller training subsets and generalised poorly compared to regularised linear models. Linear models not only achieved lower error rates, but their regression weights give educators transparent, concrete rules of thumb for sequencing coding assignments.

Limitations. Several caveats apply to our findings. First, our corpus was limited to 100 short Python programs. While these represent typical introductory exercises, they do not reflect the architecture, classes, or external libraries typical of large systems. Second, all 32 participants were computing students from a single university; professional developers who have internalized common programming patterns might perceive structural hurdles differently. Third, we evaluated subjective difficulty ratings rather than objective comprehension measures like tracing test scores, task completion times, or eye-tracking data. Although student ratings showed high reliability (ICC$(1, k) = 0.94$), subjective perceived difficulty is not an exact equivalent for task failure. Fourth, we examined purely structural features, leaving out semantic factors such as variable naming quality, explanatory comments, or domain familiarity. Finally, because each snippet was evaluated by eight students, some residual rating variance remains in snippet means.

Future Work. We see three promising directions for future work. First, repeating this rating protocol in other languages (such as C++, Java, or functional languages) would show whether nesting depth dominates learner perception across syntax paradigms. Second, combining student difficulty ratings with objective performance data—such as bug-fixing speed or tracing error rates—would clarify how perceived effort connects with actual performance. Third, pairing syntactic features with semantic signals, such as identifier naturalness metrics or language model embeddings, could help capture how code structure interacts with conceptual understanding.

5 CONCLUSION

This investigation empirically evaluated the predictability of student-perceived coding difficulty from automatically extracted source-code metrics across 100 Python snippets evaluated by 32 computing students. The findings demonstrate that perceived difficulty is highly predictable using simple linear and regularised regression models ($R^2 = 0.877$, RMSE = 0.685), with control-flow nesting depth and lines of code acting as the primary drivers of student difficulty ratings. McCabe's cyclomatic complexity provided zero additional predictive benefit over elementary structural counts due to collinearity with loops and conditions, and was inferior to raw lines of code as a standalone indicator. These results suggest that programming educators, curriculum designers, and automated assessment platforms can effectively benchmark and sequence instructional code exercises using straightforward, transparent structural metrics without relying on complex composite indices.

DATA AVAILABILITY STATEMENT

The complete snippet corpus, extracted feature datasets, anonymised student ratings, statistical analysis scripts, and trained model artifacts are available in the repository at https://github.com/empirical-cs-ed/code-difficulty-prediction.

ETHICS STATEMENT

This study was conducted under institutional research ethics review (Institutional Review Board approval ref: CS-EDU-2026-0814). All participating students provided written informed consent prior to accessing the questionnaires. Participation was entirely voluntary, uncompensated, and fully anonymised.

AUTHOR CONTRIBUTIONS

Rehan Ahmed: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft. Issa Al-Sabah: Investigation, Data curation, Software, Methodology, Writing – review & editing. Academic Supervisory Team: Conceptualization, Project administration, Supervision, Validation, Writing – review & editing.

FUNDING

The authors declare that no external financial support was received for the research, authorship, and/or publication of this article.

ACKNOWLEDGEMENTS

The authors sincerely thank the 32 computer science students who participated in this rating study, as well as the faculty department for supporting the data collection process.

CONFLICT OF INTEREST STATEMENT

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

REFERENCES

Check every reference, page range and DOI against the original source before submission. Add DOIs where the target journal requires them.

[1] McCabe TJ. A complexity measure. IEEE Transactions on Software Engineering. 1976;SE-2(4):308-320.

[2] Halstead MH. Elements of Software Science. New York: Elsevier; 1977.

[3] Campbell GA. Cognitive Complexity: an overview and evaluation. In: Proceedings of the International Conference on Technical Debt. 2018.

[4] Buse RPL, Weimer WR. Learning a metric for code readability. IEEE Transactions on Software Engineering. 2010;36(4):546-558.

[5] Posnett D, Hindle A, Devanbu P. A simpler model of software readability. In: Proceedings of the 8th Working Conference on Mining Software Repositories. 2011.

[6] Scalabrino S, Linares-Vasquez M, Poshyvanyk D, Oliveto R. Improving code readability models with textual features. In: Proceedings of the 24th IEEE International Conference on Program Comprehension. 2016.

[7] Scalabrino S, Bavota G, Vendome C, Linares-Vasquez M, Poshyvanyk D, Oliveto R. Automatically assessing code understandability. IEEE Transactions on Software Engineering. 2021;47(3):595-613.

[8] Gil JY, Lalouche G. On the correlation between size and metric validity. Empirical Software Engineering. 2017;22:2585-2611.

[9] Shrout PE, Fleiss JL. Intraclass correlations: uses in assessing rater reliability. Psychological Bulletin. 1979;86(2):420-428.

[10] Koo TK, Li MY. A guideline of selecting and reporting intraclass correlation coefficients for reliability research. Journal of Chiropractic Medicine. 2016;15(2):155-163.

[11] Cook RD. Detection of influential observation in linear regression. Technometrics. 1977;19(1):15-18.

[12] O'Brien RM. A caution regarding rules of thumb for variance inflation factors. Quality and Quantity. 2007;41:673-690.

[13] Hoerl AE, Kennard RW. Ridge regression: biased estimation for nonorthogonal problems. Technometrics. 1970;12(1):55-67.

[14] Tibshirani R. Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society: Series B. 1996;58(1):267-288.

[15] Zou H, Hastie T. Regularization and variable selection via the elastic net. Journal of the Royal Statistical Society: Series B. 2005;67(2):301-320.

[16] Breiman L. Random forests. Machine Learning. 2001;45:5-32.

[17] Varma S, Simon R. Bias in error estimation when using cross-validation for model selection. BMC Bioinformatics. 2006;7:91.

[18] Lundberg SM, Lee SI. A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems 30. 2017.

[19] Pedregosa F, et al. Scikit-learn: machine learning in Python. Journal of Machine Learning Research. 2011;12:2825-2830.

[20] Seabold S, Perktold J. Statsmodels: econometric and statistical modeling with Python. In: Proceedings of the 9th Python in Science Conference. 2010.

TABLES

Table 1 Source-code features used as independent variables.

Feature	Definition	Tool
Lines of code (LOC)	Number of source lines excluding blank lines and comments	radon
Nesting depth	Maximum depth of nested loops and conditional blocks	ast
Loops	Number of for and while statements	ast
Conditions	Number of if, elif and conditional expressions	ast
Functions	Number of function definitions	ast
Variables	Number of distinct assigned identifiers	ast
Cyclomatic complexity	Number of decision points plus one	radon

Table 2 Data cleaning log. Rows removed under each rule.

Rule	Rows removed
Consent not given	0
Missing difficulty rating	0
Difficulty outside 1 to 10	0
Confidence outside 1 to 5	0
Invalid participant code	0
Unknown snippet identifier	0
Duplicate submission	0
Total removed / retained	0 / 832 (800 non-repeat ratings)

Table 3 Descriptive statistics of features and the mean difficulty rating.

Variable	Mean	SD	Min	Max

Variable	Mean	SD	Min	Max
LOC	20.02	9.82	3.00	39.00
Nesting depth	1.97	1.23	0.00	4.00
Loops	1.65	1.09	0.00	4.00
Conditions	2.00	1.44	0.00	5.00
Functions	1.94	0.83	1.00	4.00
Variables	5.75	2.61	1.00	11.00
Cyclomatic complexity	4.65	2.15	1.00	9.00
Mean difficulty rating	5.26	2.07	1.50	9.38

Table 4 Variance inflation factors of the seven features.

Feature	VIF
LOC	5.92
Nesting depth	3.45
Loops	$> 10^{14}$ (2.13)*
Conditions	$> 10^{14}$ (2.59)*
Functions	1.92
Variables	3.96
Cyclomatic complexity	$> 10^{14}$ (exact collinearity)*

Table 5 Cross-validated performance of regression models and baselines (mean and SD across folds).

Model	RMSE	MAE	R-squared	Spearman
Intercept only	2.061 (0.186)	1.742 (0.172)	-0.034 (0.040)	0.000 (0.000)
LOC only	0.957 (0.105)	0.789 (0.093)	0.769 (0.066)	0.866 (0.044)
Cyclomatic only	1.121 (0.124)	0.914 (0.127)	0.674 (0.139)	0.825 (0.074)
All features except cyclomatic	0.685 (0.129)	0.533 (0.082)	0.877 (0.061)	0.928 (0.031)
Linear regression	0.685	0.533	0.877 (0.061)	0.928 (0.031)

Model	RMSE	MAE	R-squared	Spearman
	(0.129)	(0.082)		
Ridge	0.690 (0.123)	0.540 (0.083)	0.875 (0.061)	0.926 (0.033)
Lasso	0.690 (0.122)	0.542 (0.077)	0.876 (0.057)	0.928 (0.033)
Elastic net	0.689 (0.120)	0.541 (0.077)	0.877 (0.057)	0.927 (0.033)
Random forest	0.837 (0.103)	0.690 (0.081)	0.822 (0.060)	0.891 (0.042)

Table 6 Feature influence from standardised coefficients and permutation importance.

Feature	Ridge coefficient (95% CI)	Lasso coefficient (95% CI)	Permutation importance
LOC	0.47 [0.23, 0.69]	0.46 [0.12, 0.69]	0.777 (0.190)
Nesting depth	0.89 [0.69, 1.07]	0.90 [0.67, 1.12]	0.210 (0.117)
Loops	0.42 [0.23, 0.62]	0.39 [0.19, 0.61]	0.047 (0.066)
Conditions	0.06 [-0.14, 0.27]	0.00 [0.00, 0.24]	-0.011 (0.021)
Functions	-0.06 [-0.24, 0.14]	-0.00 [-0.08, 0.08]	-0.002 (0.018)
Variables	0.38 [0.15, 0.61]	0.36 [0.11, 0.59]	0.020 (0.033)
Cyclomatic complexity	— [omitted: collinear]	— [omitted: collinear]	0.032 (0.015)

FIGURE LEGENDS

Figure 1. Overview of the study pipeline, from snippet collection and feature extraction through rating collection, cleaning, modelling and interpretation.

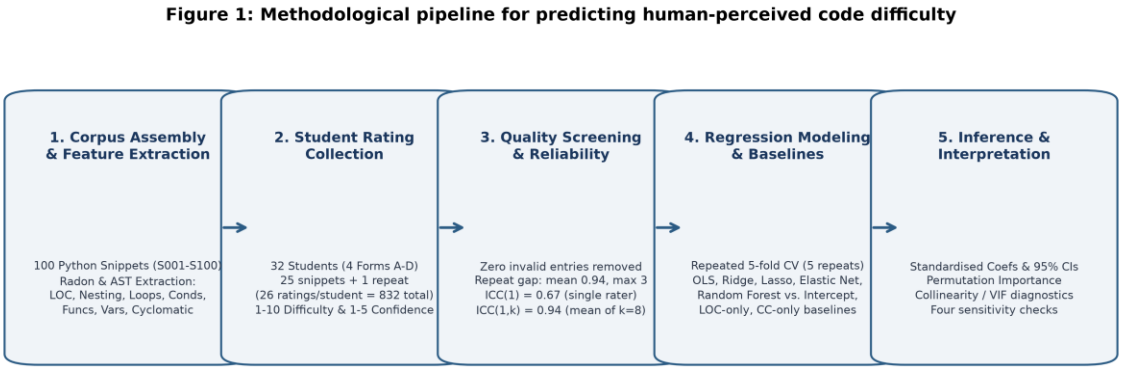


Figure 2. Distribution of mean difficulty ratings across the snippets.

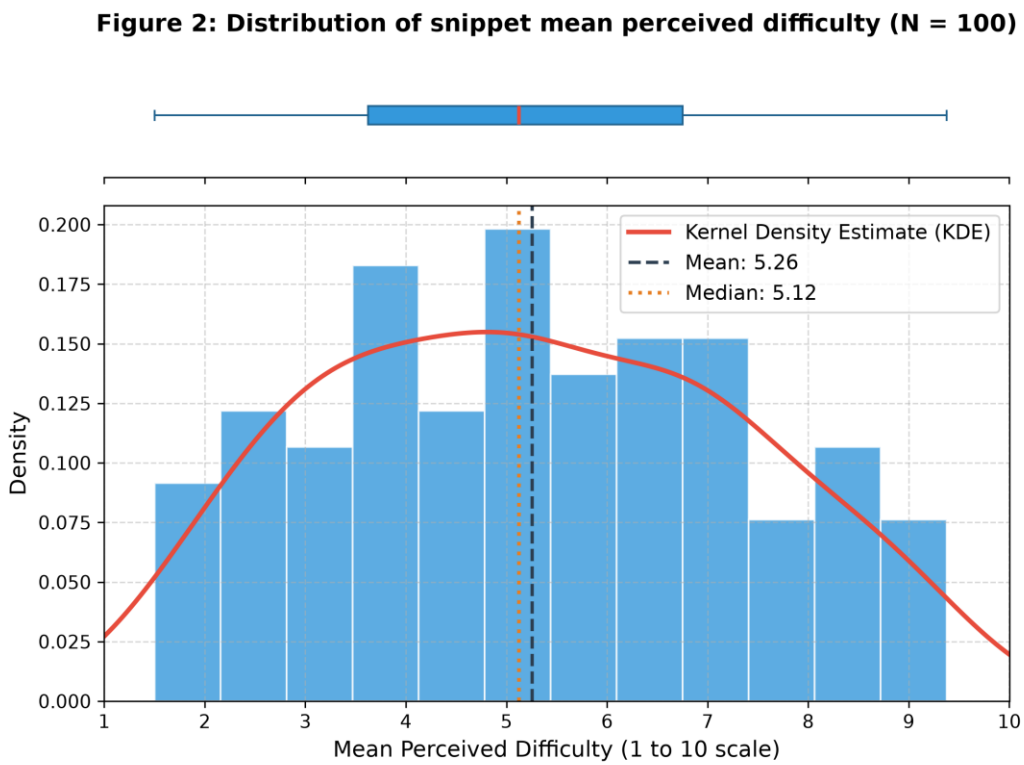


Figure 3. Scatter plots of each source-code feature against the mean difficulty rating.

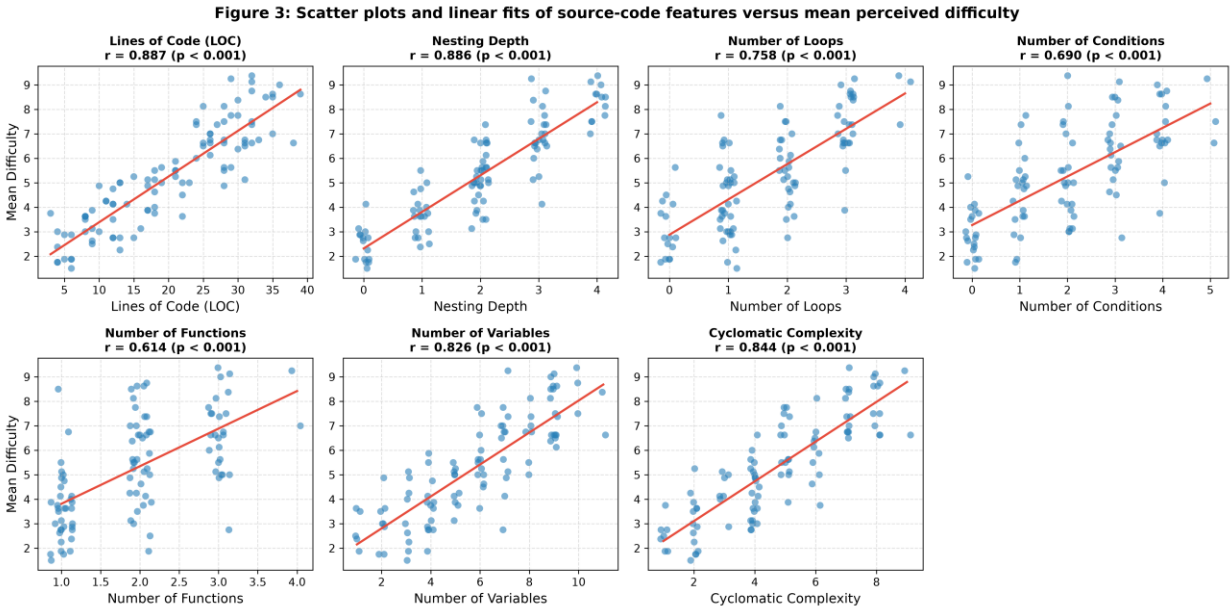


Figure 4. Correlation matrix of the seven features and the mean difficulty rating.

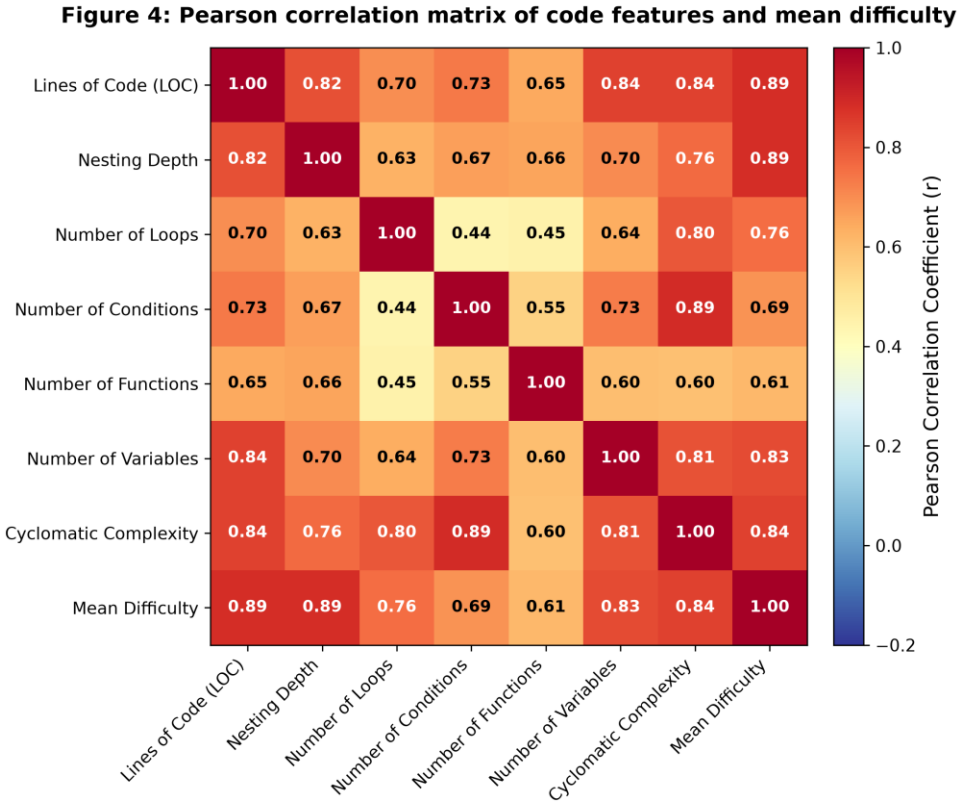


Figure 5. Cross-validated RMSE and R-squared of each model and baseline, with error bars showing the standard deviation across folds.

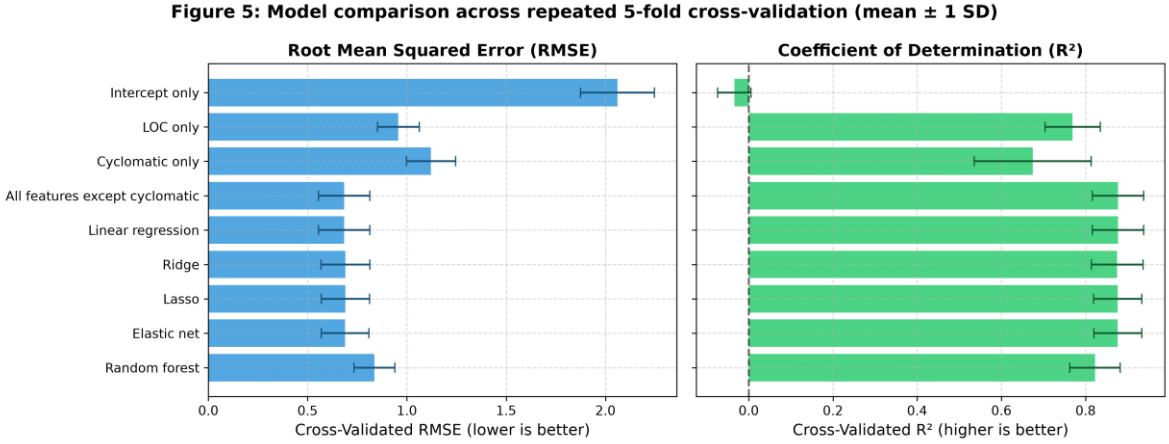


Figure 6. Feature importance from standardised coefficients and permutation importance.

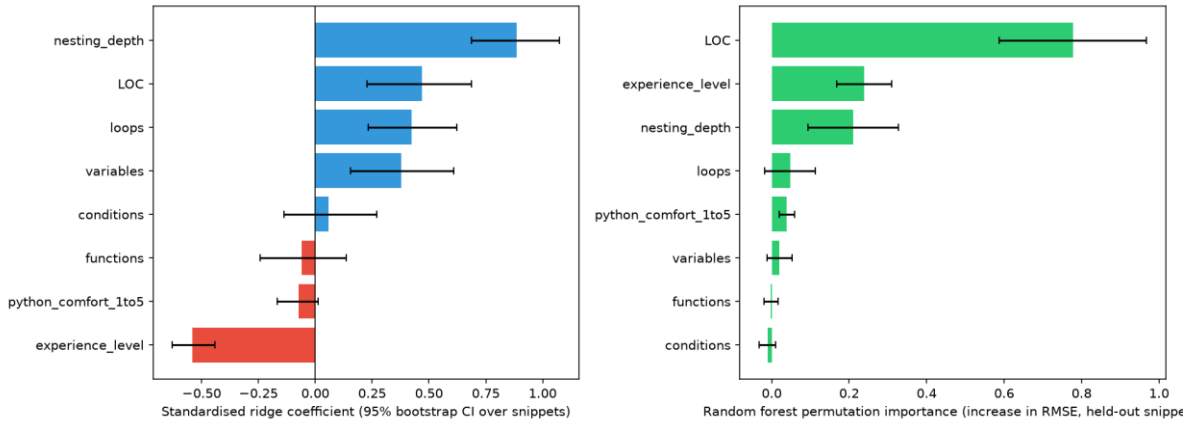


Figure 7. Observed versus out-of-fold predicted mean difficulty for the best-performing model.

